МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Харківський національний університет імені В. Н. Каразіна
Навчально-науковий інститут комп'ютерних наук
та штучного інтелекту

Звіт з виконання практичного завдання № 2
«Основи дизайну UI/UX»
1-й семестр

Назва команди: Cassiopeia Team
Назва проєкту: Cassiopeia – персональний ШІ-планувальник

Виконали: студентки групи КС43
Дмитренко Поліна Дмитрівна
Касьянова Катерина

Перевірив: викладач
Даас Тимур

Харків 2026

Практичне завдання № 2 на тему: «Інструменти дизайну, інформаційна архітектура, UI-kit та прототип»

Мета: дослідити сучасні інструменти для дизайну, спроєктувати структуру та ключові сценарії використання Cassiopeia, а потім – уже на цій основі – зібрати інтерактивне рішення з базовим UI-kit. Простіше кажучи, вимоги з PRD, які в ПЗ № 1 існували лише як текст, тут мають перетворитися на щось, що можна побачити, клікнути й перевірити.

Обраний шлях виконання Частини 4: Варіант В – імплементація в коді (AI-first), де UI-kit оформлений як Storybook.

Склад команди та ролі з ПЗ № 1 не змінилися, тож окремо їх не повторюємо.

Частина 1 – Аналіз інструментів для дизайну

Почнемо з того, що саме нам потрібно. Cassiopeia – це не лендинг і не набір красивих картинок, а застосунок, у якому користувач веде діалог, отримує план і постійно його змінює. Отже, інструмент має вміти три речі: тримати систему компонентів зі станами, давати справжню (а не уявну) навігацію між екранами і, бажано, не змушувати нас робити ту саму роботу двічі, коли в ПЗ № 3 дійде до коду.

З цими вимогами ми й підійшли до порівняння. В основну таблицю потрапили чотири інструменти: Figma як класика, без якої подібний аналіз просто неповний, і три AI-інструменти, що генерують уже не макет, а робочий інтерфейс – v0, Lovable та Claude Code.

Таблиця 1. Порівняння інструментів для прототипування

Критерій	***Figma (+ Figma Make)***	***v0 (Vercel)***	***Lovable***	***Claude Code***
Тип результату	Макети та клікабельні прототипи; Figma Make – prompt-to-app:	React-код за текстовим запитом; працює з Next.js,	Вебзастосунок за текстовим запитом (нові проєкти – TanStack Start,	Агент, що читає наявний код, редагує файли й виконує

Критерій	*Figma (+ Figma Make)*	*v0 (Vercel)*	*Lovable*	*Claude Code*
	інтерактивний прототип за текстовим запитом	Tailwind, shadcn/ui	старіші – React + Vite)	команди прямо в репозиторії
UI-kit / компоненти	Компоненти, варіанти, змінні (design tokens), бібліотеки стилів	Генерує React-компоненти на основі shadcn/ui	Компоненти створюються в коді проєкту разом зі сторінками	За текстовою інструкцією може створити tokens, компоненти та Storybook-стори в наявному проєкті
Інтерактивність	Переходи між фреймами; у Figma Make – інтерактивні прототипи з анімаціями	Робочий код із preview; деплой на Vercel в один клік	Робочий застосунок із preview	Справжня логіка в коді, локальний запуск
Командна робота	Сильна: спільне редагування, коментарі, Dev Mode	Можна відкрити pull request для рев'ю	Git-синхронізація з GitHub, GitLab, Bitbucket	Працює з git: коміти, гілки, pull requests
Поріг входу	Програмування не потрібне	Запит природною мовою; для доопрацювання коду – знання React	Запит природною мовою; редагування коду вручну – лише на платному тарифі	Потрібні базові навички роботи з терміналом або IDE та Git
Безкоштовний доступ	Безкоштовно для індивідуальних користувачів (3 проєкти); Professional – $12/міс. за редактора при оплаті за рік	Free – $5 кредитів на місяць; Premium – $20 кредитів на місяць	Free – 5 кредитів на день, але не більше 30 на місяць	Безкоштовного плану немає: входить у підписки Claude Pro, Max та Ultra
Головний ризик для Cassiopeia	Дизайн доведеться окремо реалізовувати в коді в ПЗ № 3	Орієнтація на екосистему Vercel / Next.js	Не можна підключити наявний репозиторій – Lovable створює новий	Потрібно перевіряти та структурувати згенерований код

Тарифи та можливості наведено за офіційними сторінками й документацією інструментів станом на листопад 2026 р. (тарифи можуть змінюватися); посилання – наприкінці цієї частини.

Якщо подивитися на таблицю уважніше, видно цікаву річ. Figma найсильніша там, де йдеться про сам дизайн: компоненти, варіанти, змінні, спільна робота. Проте результат роботи в ній – макет, і код для ПЗ № 3 все одно доведеться писати окремо. Figma Make вже вміє генерувати інтерактивні прототипи за запитом, але це окремий файл у Figma, а не наш проєкт. v0, Lovable і Claude Code одразу працюють з кодом, і різниця між ними – радше в тому, скільки контролю над результатом залишається в команди та чи можна продовжувати роботу в уже наявному репозиторії.

Альтернативні інструменти

Окрім основних, ми розглянули ще кілька інструментів. Коротко про кожен:

Таблиця 2. Альтернативні інструменти

Інструмент	***Тип***	***Для чого підходить***	***Чому не обрали для Частини 4***
Canva	Класичний	Презентації, банери, соцмережі	Орієнтована на графіку й презентації, а не на UI-kit і прототипування застосунку
Miro / FigJam	Класичний (дошки)	Дошки для sitemap, user flow, мудбордів, воркшопів	Не призначені для екранів і UI-kit; схеми в Частині 2 ми побудували в draw.io
Uizard	ШІ	Autodesigner – макети з текстового опису (на тарифі Pro); Screenshot Scanner і Wireframe Scanner – макети зі скріншотів та ескізів	Результат – макет, а не код нашого проєкту
Google Stitch	ШІ (експеримент Google Labs)	UI-дизайни та фронтенд-код з тексту або зображень; дизайн можна вставити у Figma	Генерує окремі екрани, а не Storybook і структуру нашого проєкту
Framer AI	ШІ	AI-агенти для створення сайтів, лендингів і портфоліо	Орієнтований на вебсайти, а не на застосунок з логікою

Окремо скажемо про Google Stitch і Uizard. Вони зручні для швидкого пошуку ідей: за текстовим описом або ескізом Google Stitch за кілька хвилин генерує кілька варіантів дизайну. Але результат – окремі екрани, а для Частини 4 нам потрібна єдина система компонентів зі станами.

Оцінка інструментів для Cassiopeia

Щоб вибір не виглядав «на око», оцінюємо чотири основні інструменти за п’ятибальною шкалою за критеріями, важливими саме для нашого проєкту.

Таблиця 3. Оцінка інструментів (1 – погано, 5 – відмінно)

Критерій	***Figma***	***v0***	***Lovable***	***Claude Code***
UI-kit і компоненти зі станами	5	4	3	5
Реальна інтерактивність і навігація	3	4	4	5
Повторне використання в ПЗ № 3	1	3	2	5
Контроль над структурою коду	1	3	2	5
Поріг входу для команди	4	4	4	3
Безкоштовний доступ	4	3	2	1
Разом (з 30)	18	21	17	24

Найбільше балів набрав Claude Code (24 з 30). Він випереджає решту там, де для нас важливіше за все: компоненти зі станами, справжня навігація, повторне використання коду в ПЗ № 3 і контроль над структурою. Програє він лише за безкоштовним доступом і порогом входу, адже потрібна підписка та базова робота з терміналом. Figma, навпаки, найдоступніша, але її макет у ПЗ № 3 доведеться реалізовувати в коді заново.

Обраний інструмент та обґрунтування

Цю роботу зроблено за допомогою Claude Code: він створив design tokens, компоненти зі stories для Storybook та екрани прямо в репозиторії Cassiopeia.

Наш вибір – Варіант В та Claude Code. Аргумент тут, власне, один, зате вагомий: робоча версія інтерфейсу Cassiopeia в коді в нас уже є. Малювати ті самі екрани заново у Figma лише заради того, щоб у ПЗ № 3 знову повертатися до коду, – це подвійна робота без жодної користі для продукту. Тож ми розвиваємо те, що є, а UI-kit, створений зараз, одразу стає фундаментом для наступного етапу.

Уточнимо, що саме ми «розвиваємо». UI-kit і прототип зроблено як новий фронтенд у папці ui-kit/ на основі палітри та екранів наявного застосунку (FastAPI із шаблонами). До бекенду й справжнього ШІ-агента його буде підключено в ПЗ № 3.

Що конкретно нам дає цей інструмент? По-перше, він працює прямо в репозиторії: за текстовою інструкцією створює design tokens, компоненти та стори для Storybook. По-друге, навігація тут справжня, тому user flow з Частини 2 можна пройти від першого екрана до останнього, а не лише «клікнути по картинках». І ще один плюс, про який легко забути, – він не прив’язує нас ні до конкретного хостингу, ні до фреймворку.

Без недоліків, звісно, не обійшлося. Потрібна платна підписка, а також базові навички роботи з терміналом і Git – для когось це може стати бар’єром. Згенерований код обов’язково треба перевіряти, зокрема чи екрани справді використовують компоненти з UI-kit, а не дублюють стилі. Нарешті, візуального редактора немає, тож навіть дрібну правку дизайну доводиться робити через код або черговий запит.

Джерела:

- Figma – тарифи
- Figma Make

- v0 – тарифи
- Lovable – тарифи
- Claude Code
- Google Stitch
- Uizard
- Figma Make – доступ і ліміти
- v0 – документація
- v0 – оновлені тарифи (кредити)
- Lovable – плани та кредити
- Lovable – FAQ (стек, Git-синхронізація)
- Claude Code – документація
- Claude Code – доступ за планами Pro, Max, Ultra
- Google Stitch – анонс (Google Developers Blog)
- Uizard – AI-функції
- Framer AI

Частина 2 – User flow та інформаційна архітектура

Структуру застосунку ми виводили з функцій MVP, описаних у PRD. Головне рішення, на якому тримається вся архітектура, – центр Cassiopeia не меню і не календар, а чат. Майже будь-яка дія (додати справу, перенести її, змінити пріоритет чи навіть оцінити сферу життя в Колесі балансу) починається з розмови. Решта розділів, по суті, показує результат цієї розмови в зручному, структурованому вигляді. Схеми user flow побудовано в draw.io, а sitemap зведено за реальною навігацією прототипу з Частини 4.

Sitemap

Sitemap вийшов трирівневим. Нагорі – сам застосунок, під ним – шість розділів нижньої навігації (Today, Week, Month, Year, Wheel, Chat), а

всередині розділів – окремі екрани та стани. Vision & Mission і Task Tracker відкриваються з Wheel of Balance, а деталі дня (Day) – з Week і Month.

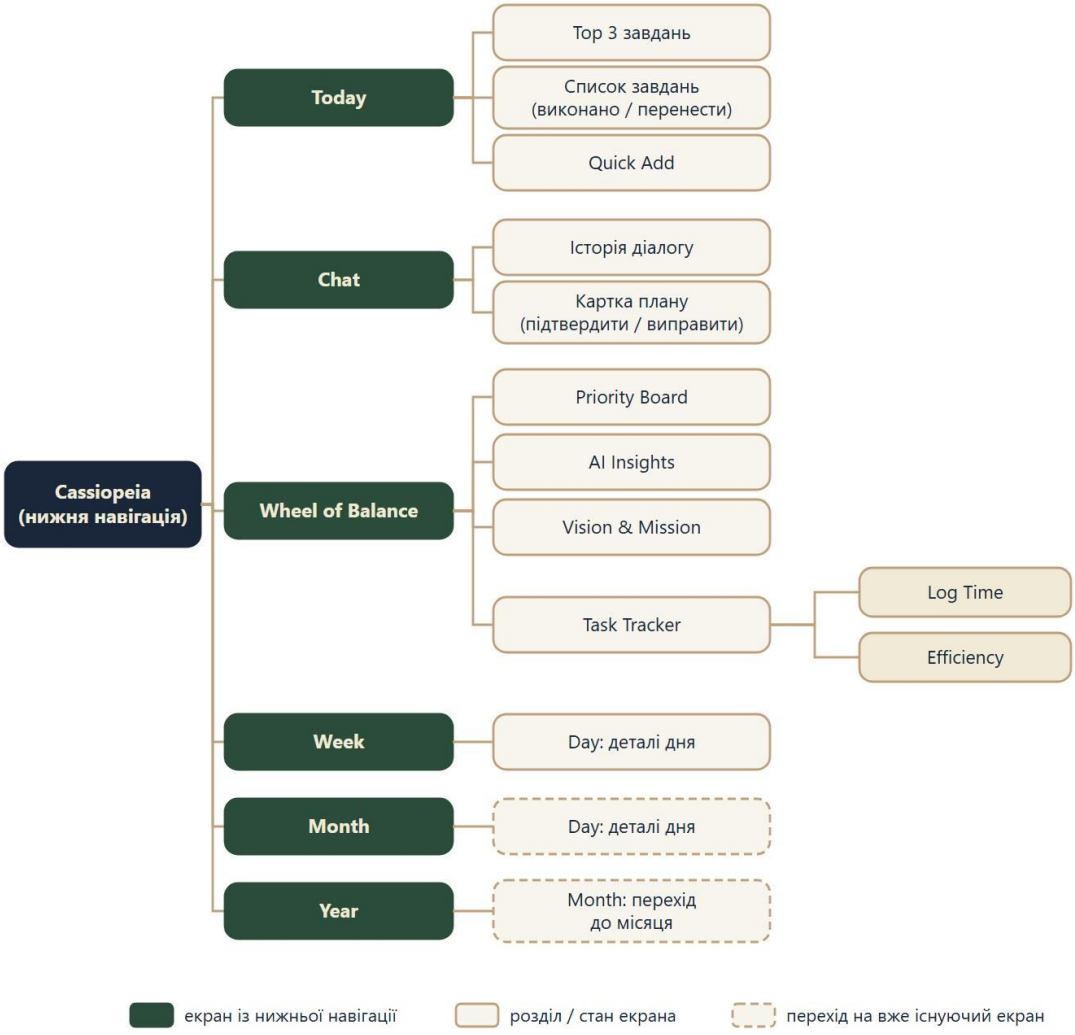


Рисунок 1. Sitemap Cassiopeia

Ядро продукту – Chat та Today: з ними користувач працює щодня. У чаті справи з’являються, на Today – стають планом із Top 3. Week, Month і Year потрібні, щоб бачити день у контексті тижня, місяця й року: саме тут видно справи, які не вмістилися в реалістичний план і були перенесені. Wheel of Balance та Vision & Mission відкриваються рідше, проте саме вони дають системі контекст – що для людини важливо, – без якого вибір Top 3 зводився б до звичайного сортування за дедлайном.

Щоб було зрозуміло, як між цими розділами рухатися, ми окремо розписали точки входу та основні дії:

Таблиця 4. Навігація між розділами

Розділ	***Як потрапити***	***Основні дії***
Chat	Нижня навігація; кнопка «Plan with chat» на Today	Описати справи, перевірити картку плану, виправити назви, підтвердити
Today	Нижня навігація; автоматично після підтвердження плану	Переглянути Top 3, відмітити виконане, перенести завдання (з Undo), швидко додати справу
Week	Нижня навігація	Переглянути сім днів із завданнями
Month	Нижня навігація; плитка місяця на екрані Year	Обрати день у календарі й переглянути його завдання (Day)
Year	Нижня навігація	Побачити навантаження за місяцями, перейти в потрібний місяць
Wheel of Balance	Нижня навігація	Переглянути радар, Priority Board, AI-інсайти; перейти у Vision & Mission чи Task Tracker
Vision & Mission / Task Tracker	Кнопки на екрані Wheel	Редагувати текст «чому я планую»; переглянути ефективність за часом; «Back to Wheel»

Зверніть увагу: до чату веде кілька шляхів – з нижньої навігації та з кнопки «Plan with chat» на Today, зокрема й з порожнього стану цього екрана. Це навмисно. Якщо щось пішло не за планом, користувач має дістатися до перепланування в один дотик, а не шукати потрібний розділ.

User flow головного сценарію

Головний сценарій ми для себе назвали «від хаосу в голові до плану дня». Суть проста: людина одним повідомленням вивантажує все, що тримає в голові, і отримує реалістичний план, який потім можна змінити, якщо день піде інакше. Позначення на схемі стандартні – прямокутники відповідають екранам і діям, ромби – рішенням, а зелені овали – кінцевим станам.

Схема нижче описує цільовий сценарій повної версії продукту. У прототипі Частини 4 реалізовано його центральну частину, про що сказано після опису розгалужень.

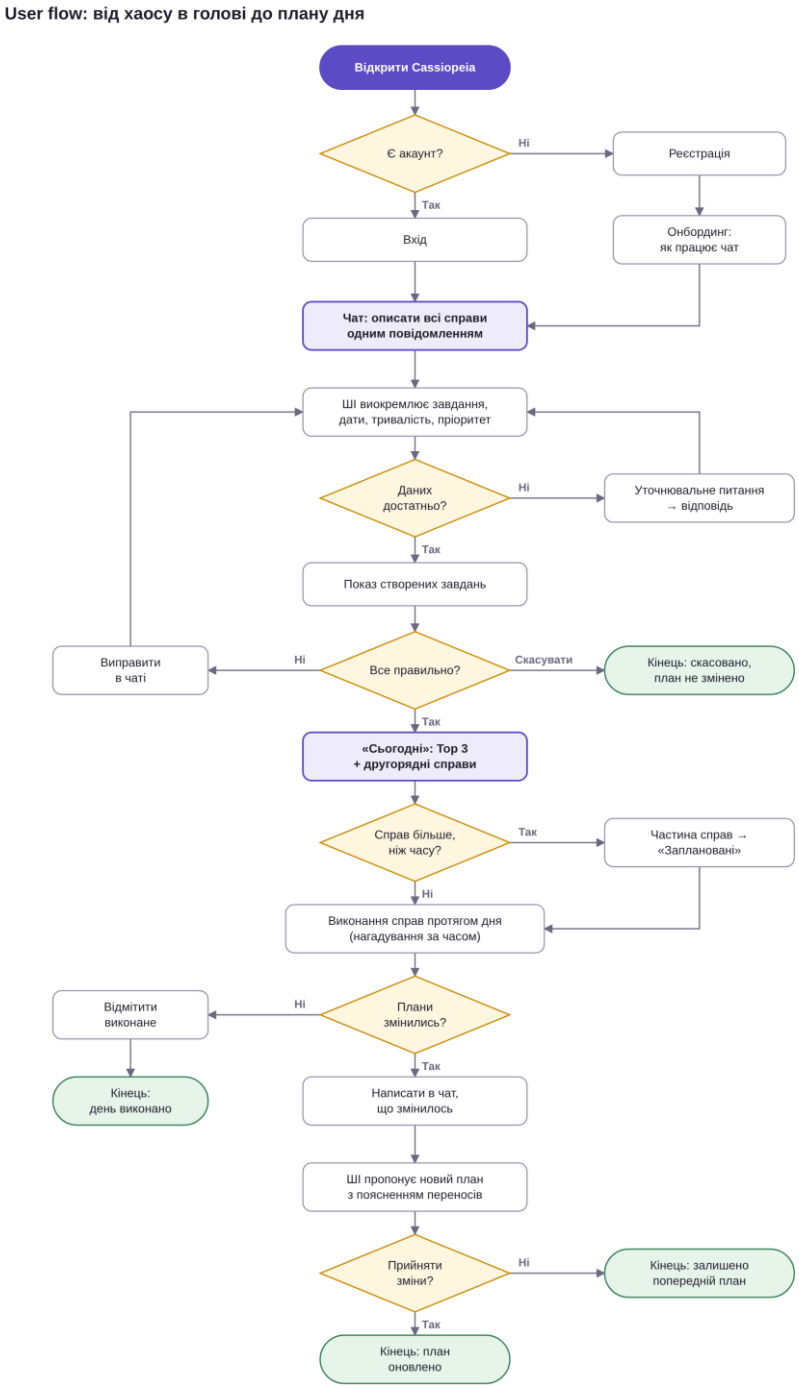


Рисунок 2. User flow «від хаосу в голові до плану дня» (цільовий сценарій)

Схема виглядає об’ємною, але насправді кожне розгалуження тут відповідає конкретному UX-принципу з ПЗ № 1. Пройдемося по них.

Перше розгалуження – на вході. Новий користувач спершу реєструється і проходить коротенький онбординг (буквально пояснення, що з чатом можна говорити звичайною мовою), а той, хто вже має акаунт, потрапляє в чат одразу.

Далі – момент, який ми вважаємо одним із найважливіших. Якщо ШІ бракує даних, скажімо, незрозуміло, «у п’ятницю» – це ця чи наступна, він не вгадує, а ставить коротке уточнювальне питання. Помилка в даті коштує користувачу значно дорожче, ніж одне зайве повідомлення.

Коли завдання створено, людина бачить їх і вирішує сама: підтвердити, виправити в чаті чи взагалі скасувати. В останньому випадку план просто не змінюється – так реалізовано принцип «можливість скасування». Схожа логіка і з перевантаженням: якщо справ більше, ніж реально встигнути, частина переїжджає в «Заплановані», а не зникає безслідно.

Ну і, мабуть, найжиттєвіша гілка – коли плани змінюються посеред дня. Користувач пише про це в чат, ШІ пропонує новий розклад і пояснює, чому що переносить. Але застосовуються зміни лише після підтвердження: контроль, як ми й обіцяли в PRD, залишається за людиною.

У підсумку сценарій має чотири кінцеві стани: «день виконано», «план оновлено», «залишено попередній план» і вихід зі скасуванням ще на етапі перевірки завдань.

У прототипі реалізовано центральну частину цього сценарію: від чату через картку плану до екрана Today, а також виконання й перенесення завдання з можливістю скасування (Undo). Реєстрація, онбординг, розділ «Заплановані», уточнювальні запитання та перепланування через чат у прототип не входять, а перенесені справи користувач бачить у Week на наступний день. Проходження сценарію крок за кроком описано в Частині 4.

Додатковий user flow: оцінка Колеса балансу

Вимоги просили мінімум один flow, але ми вирішили додати ще й другий. Причина в тому, що без нього незрозуміло, як Колесо балансу пов’язане з денним плануванням, – а саме цей зв’язок відрізняє Cassiopeia від звичайного таск-менеджера.

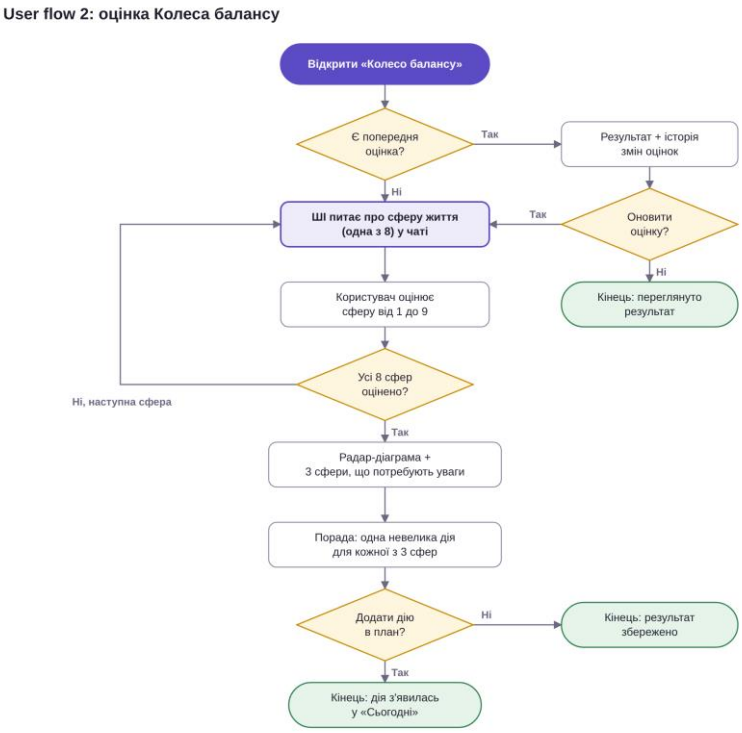


Рисунок 3. User flow «оцінка Колеса балансу» (цільовий сценарій)

Тут є цикл: ШІ по черзі питає про кожну з восьми сфер, а користувач ставить оцінку від 1 до 9. Повторний користувач іде іншою гілкою – спочатку бачить попередній результат з історією змін і вже тоді вирішує, чи оновлювати оцінку. Найцікавіше відбувається наприкінці. Отримавши радар-діаграму та поради для трьох найслабших сфер, людина може одразу додати запропоновану дію в план на сьогодні, і таким чином рефлексія перетворюється на конкретний крок.

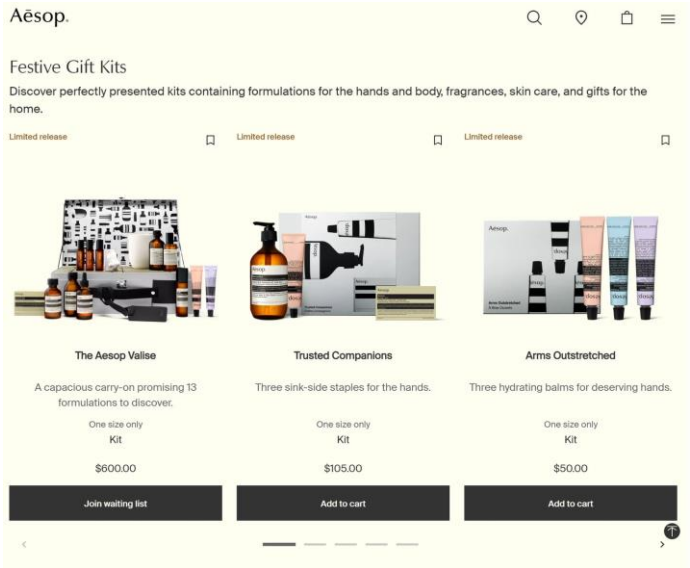
У прототипі реалізовано результат цього сценарію: радар із вісьмома сферами на шкалі 1–9 і Priority Board із трьома найслабшими сферами та

підказкою дії. Послідовний діалог оцінювання й історія змін у прототип не входять.

Частина 3 – Мудборд

Збираючи мудборд, ми відштовхувалися від головної ідеї продукту – менше розумового навантаження. Якщо застосунок обіцяє зняти хаос із голови, то сам він точно не має виглядати хаотично. Тому шукали інтерфейси спокійні, з великою кількістю повітря, чіткою ієрархією й мінімумом елементів на екрані. Стиль Cassiopeia ми визначили як «Old Money»: теплий кремовий фон, глибокий синій, темно-зелений і золотий акцент, а для заголовків – serif із характером. Перші три референси задають настрій і типографіку, решта п’ять – це робочі інтерфейси планувальників і чату, з яких ми беремо структуру екранів, але не їхню яскраву палітру.

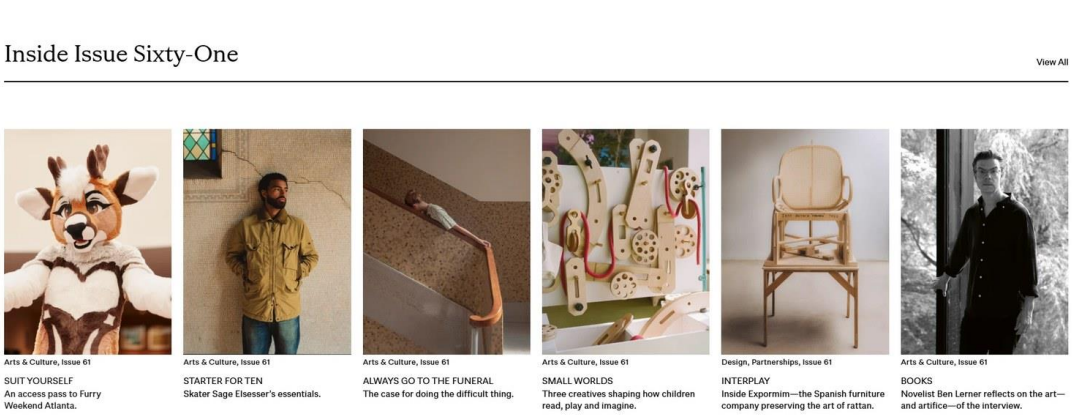
1. Aesop (aesop.com)



Світлий кремовий фон і дуже багато повітря: товар стоїть у центрі, а інтерфейс «мовчить». Заголовки легкі, основний текст – чистий гротеск, кнопки – темні прямокутні на всю ширину картки, а службові мітки (наприклад, «Limited release») – приглушеного бурштинового кольору. Від

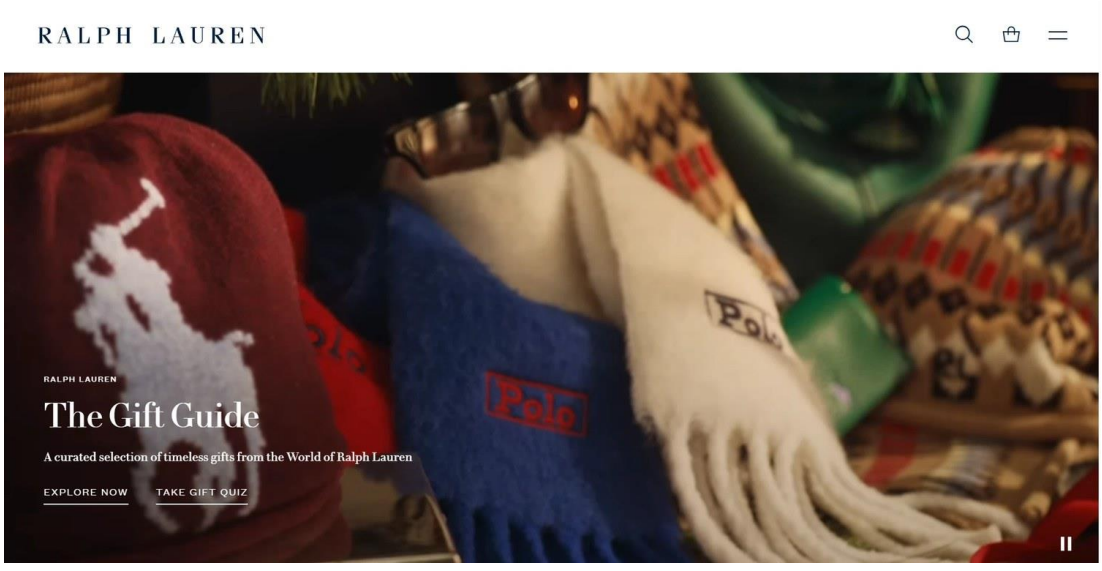
Aesop ми беремо відчуття дорогої тиші: кремовий фон, великі відступи, стримані темні кнопки.

2. Kinfolk (kinfolk.com)



Редакційна подача: serif-заголовок із тонкою лінією-розділювачем, дрібний рядок категорії над назвою матеріалу, акуратна сітка однакових карток. Звідси – типографічна ієрархія й тонкі лінії між секціями, які ми використовуємо в заголовках екранів.

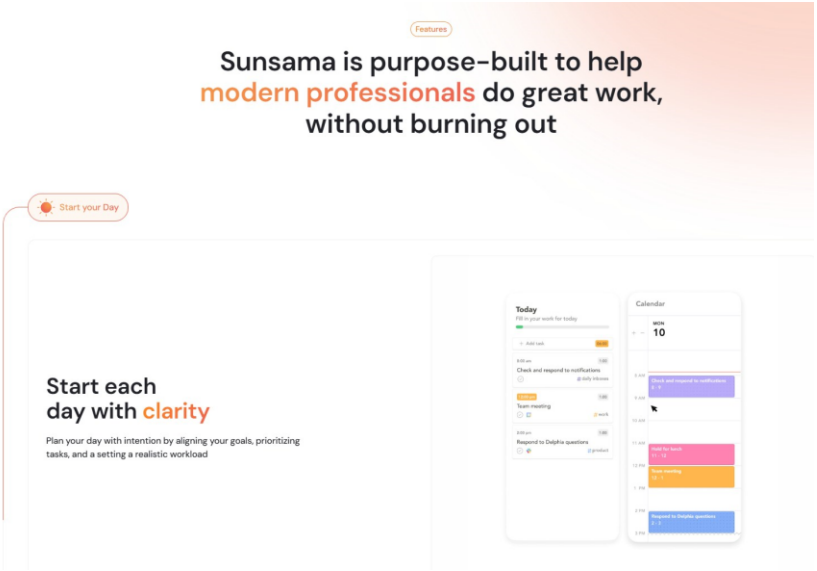
3. Ralph Lauren (ralphlauren.com)



Класичний serif-логотип із великим міжлітерним інтервалом, великі serif-заголовки та посилання великими літерами з підкресленням замість звичайних кнопок. Ми беремо serif-заголовки та дрібні мітки великими

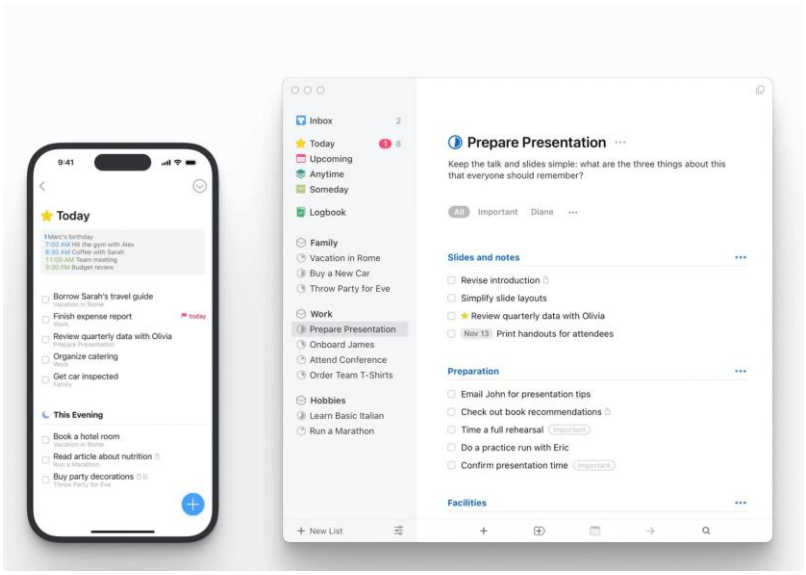
літерами з розрядкою над заголовком (у UI-kit це стиль eyebrow), а також поєднання темно-синього з теплими акцентами.

4. Sunsama (sunsama.com)



Екран «Сьогодні» з картками завдань, де видно час початку й тривалість, кружечком-чекбоксом, смугою прогресу дня й календарем-таймлайном поруч. Для Cassiopeia це орієнтир структури картки завдання та екрана Today. Яскраві кольори ми не беремо.

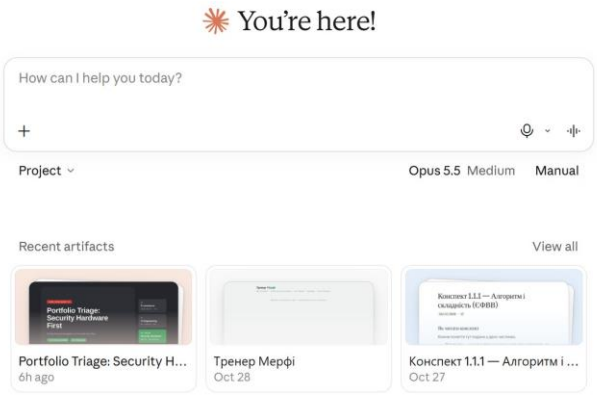
5. Things 3 (culturedcode.com/things)



Мінімалістичні списки з прямокутними чекбоксами, дрібним сірим підписом проєкту під назвою завдання, групами за часом дня (наприклад,

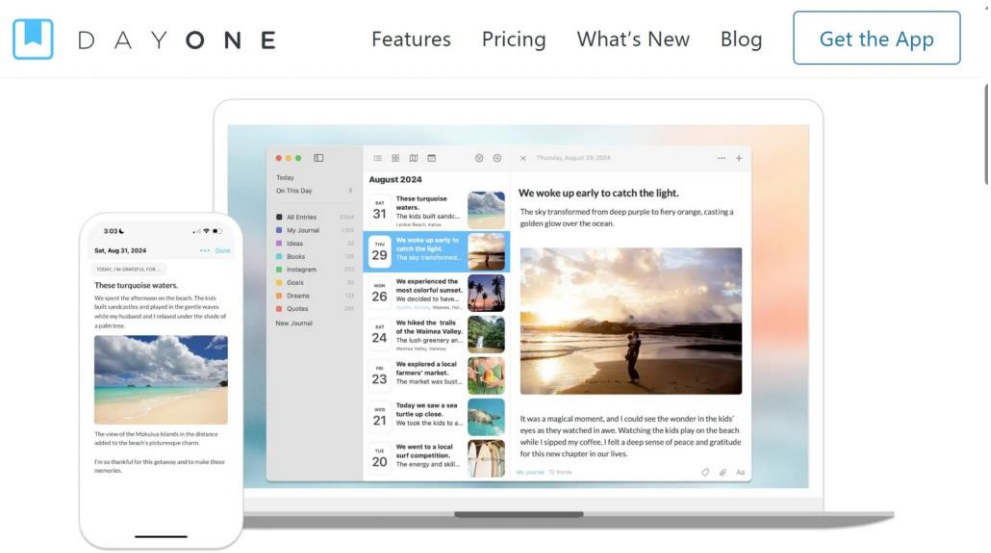
«This Evening») і круглою кнопкою «+». Беремо вигляд списку завдань, групування й швидке додавання.

6. Claude (claude.ai)



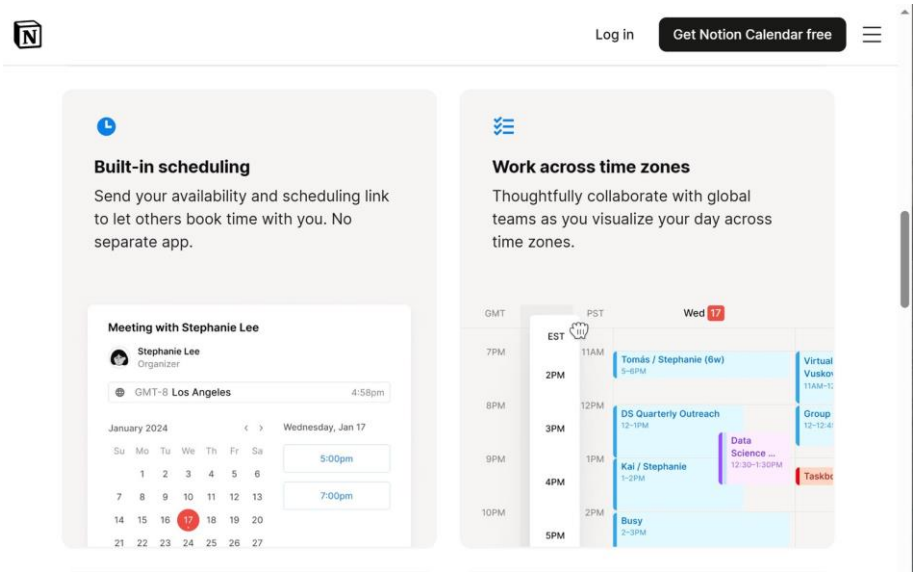
Привітання serif-шрифтом по центру, великий закруглений інпут як єдиний фокус екрана й акуратні картки з мініатюрами нижче. Оскільки головний екран у нас – чат, без цього референсу не обійтися: одне велике поле введення й мінімум елементів – люди вже звикли до такого формату, і ламати цю звичку немає сенсу.

7. Day One (dayoneapp.com)



Список записів із великою датою зліва (день тижня й число) та широкою зоною читання; інтерфейс відступає на другий план. Беремо блок дати для екранів Week і Day та спокійну зону тексту для Vision & Mission.

8. Notion Calendar (notion.com/calendar)



Чисті сітки місяця й тижня, виразна позначка сьогоднішнього дня (червоне коло), події з кольоровою смужкою зліва та пластини-картки з жирними заголовками. Беремо позначку «сьогодні» й чисту сітку для екранів Week і Month.

Джерела: aesop.com, kinfolk.com, ralphlauren.com, sunsama.com, culturedcode.com/things, claude.ai, dayoneapp.com, notion.com/calendar (скріншоти зроблено в листопаді 2026 р.).

Висновок мудборду

Якщо звести все разом, вийде така картина:

Таблиця 5. Що ми беремо з мудборду в Cassiopeia

Референс	***Що беремо***	***Де використовуємо***
Aesop, Kinfolk	Кремовий фон, повітря, serif-заголовки, тонкі лінії-розділювачі	Загальний фон, заголовки екранів, картки
Ralph Lauren	Serif-заголовки, дрібні великі літери з розрядкою, поєднання navy з теплими акцентами	Primary navy, стиль eyebrow над заголовком
Sunsama	Картка завдання з часом і тривалістю, смуга прогресу дня	Екран Today, компоненти TaskItem і ProgressBar
Things 3	Чекбокси, мінімалістичні списки, групування за часом	TaskItem, списки на Week і Month
Claude	Привітання, велике поле введення, мінімум елементів	Екран Chat, компонент Input
Day One	Блок дати, широка зона читання	Day, Vision & Mission
Notion Calendar	Чиста сітка, позначка «сьогодні»	Екрани Week і Month

Виявилося, що при всій різноманітності в цих прикладів чимало спільного. Світлий фон і лише один-два виразні акцентні кольори. Чіткий гротеск для тексту й виразний serif для заголовків. Великі відступи, м’які заокруглення карток і кнопок, мінімум елементів на екрані. Саме ці рішення ми й переносимо в UI-kit: темно-синій стає primary, темно-зелений – secondary, нейтральні кремові відтінки відповідають за фон і поверхні, а для успіху, помилки та попередження заведено окремі семантичні кольори. Золотий акцент ми приберегли для виділення головного: фон картки Top 3, мітки часу й дрібні заголовки над розділами, щоб головні справи дня справді впадали в око.

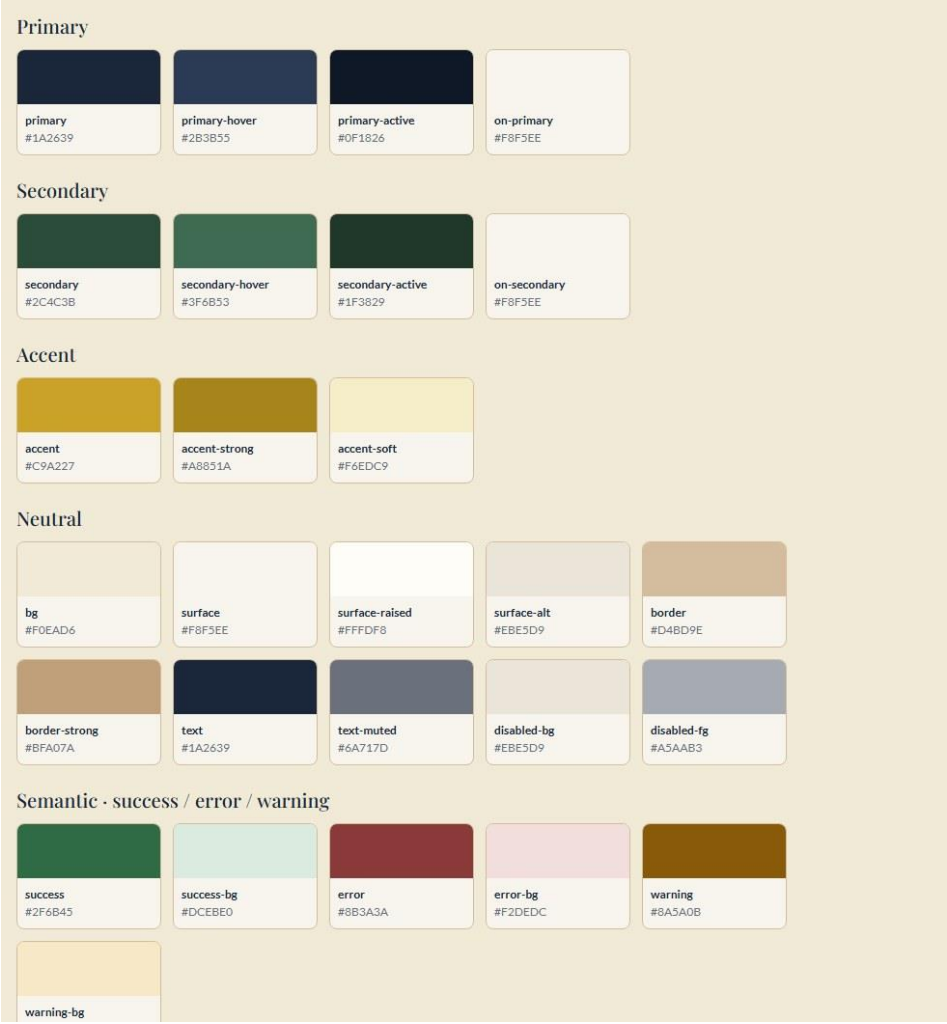


Рисунок 4. Палітра Cassiopeia, виведена з мудборду (Storybook, Foundations → Colors)

Частина 4 – UI-kit та прототип екранів (Варіант B)

Обрано шлях B: UI-kit і ключові екрани реалізуються в коді за допомогою AI-інструменту, а UI-kit оформлюється у Storybook, де кожен компонент має окрему сторі зі своїми станами.

Результат Частини 4 – працююче рішення в репозиторії; нижче наведено посилання, стек і порядок запуску.

Посилання на репозиторій: https://github.com/Katrusevich/Cassiopeia (код UI-kit – у папці ui-kit/).

Стек: React 19, Vite 8, Storybook 10, іконки lucide-react. Стилі – чистий CSS без фреймворків, побудований на design tokens; мова – JavaScript.

Інструкція запуску:

1. Клонувати репозиторій: git clone https://github.com/Katrusevich/Cassiopeia.git
2. Перейти в папку UI-kit і встановити залежності: cd ui-kit, потім npm install
3. Запустити Storybook: npm run storybook (відкриється на http://localhost:6006)
4. Запустити клікабельний прототип: npm run dev (відкриється на http://localhost:5173)

Прототип також доступний усередині Storybook як історія Screens → Prototype. Для статичної збірки Storybook є команда npm run build-storybook.

Як ми працювали

Порядок роботи був таким. Спочатку ми винесли всі візуальні значення в один файл tokens.css, і лише потім писали компоненти, а після них – екрани. Правило для Claude Code було одне: жодних кольорів чи відступів у самих компонентах, лише змінні з tokens.css. Дотримання перевірили пошуком по коду: hex-коди зустрічаються тільки в tokens.css. Кожен компонент одразу супроводжується stories у Storybook, а екрани складаються виключно з цих компонентів. Результат перевірено в браузері: пройдено весь сценарій від чату до перенесення завдання.

UI-kit

Design tokens ми не вигадували окремо – їх виведено з мудборду й палітри застосунку, що вже існувала (рисунок 4), тож UI-kit і мудборд говорять однією візуальною мовою. Усі значення живуть в одному файлі

tokens.css у три шари: primitives (сира палітра), semantic (ролі: primary, surface, text, success…) і themes (перепризначення ролей). Компоненти посилаються лише на семантичні токени, тому змінити вигляд продукту можна в одному місці.

Таблиця 6. Design tokens

Група	***Токени***	***Значення***
Primary	--color-primary, -hover, -active, --color-on-primary	Navy #1A2639 → #2B3B55 → #0F1826; текст на ньому #F8F5EE
Secondary	--color-secondary, -hover, -active	Forest green #2C4C3B → #3F6B53 → #1F3829
Accent	--color-accent, -strong, -soft	Золото #C9A227, #A8851A, світлий фон #F6EDC9
Neutral	--color-bg, surface, surface-alt, border, text, text-muted, disabled-*	Кремові #F0EAD6 / #F8F5EE / #EBE5D9, camel #D4BD9E, сірий текст #6A717D
Semantic	--color-success, error, warning (+ -bg)	#2F6B45, #8B3A3A, #8A5A0B на приглушених фонах
Типографіка	--font-heading, --font-body, --fs-h1 … --fs-caption	Playfair Display / Lato; H1 28–36 px, H2 24, H3 20, body 16, small 14, caption 12
Відступи	--space-1 … --space-8	Крок 4 px: 4, 8, 12, 16, 24, 32, 48, 64
Форма	--radius-sm/md/lg/xl, --shadow-sm/md	8, 15, 20, 30 px; м’які тіні кольору navy

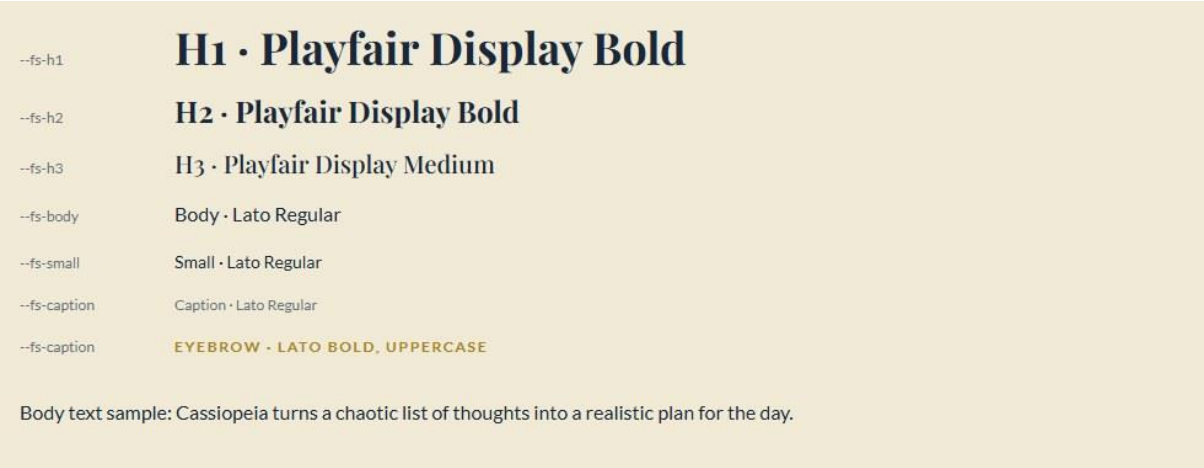


Рисунок 5. Типографічна шкала: H1–H3, body, small, caption, eyebrow (Storybook, Foundations → Typography)

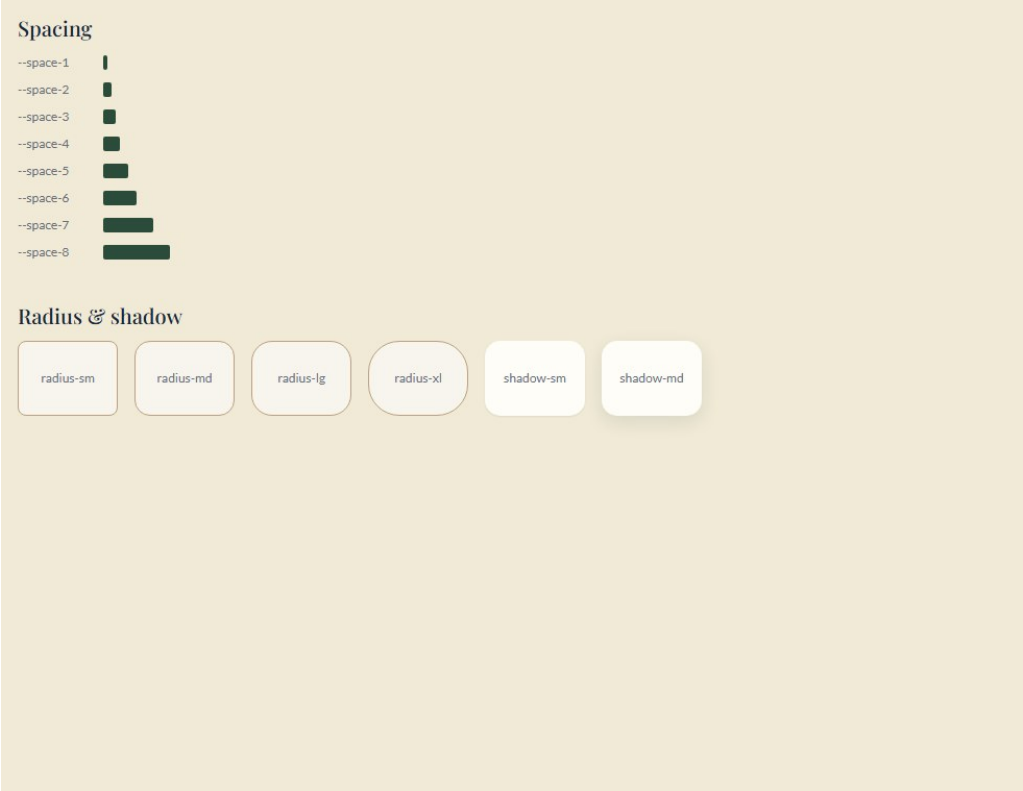


Рисунок 6. Відступи, радіуси та тіні (Storybook, Foundations → SpacingAndShape)

Щоб компоненти поводилися передбачувано, для всіх станів діють спільні правила:

Таблиця 7. Правила станів компонентів

Стан	***Правило***
default	primary – фон navy #1A2639, текст #F8F5EE; secondary – фон forest green #2C4C3B; outline – рамка navy на прозорому фоні; ghost – без рамки; danger – #8B3A3A
hover	primary #2B3B55; secondary #3F6B53; outline і ghost – фон #EBE5D9; danger яскравішає; у поля рамка стає navy; в інтерактивної картки – рамка navy і тінь
active	primary #0F1826; secondary #1F3829; кнопка зміщується на 1 px вниз
focus	обведення 2 px золотого кольору #C9A227 з відступом 2 px; у поля – рамка #A8851A і світлий ореол #F6EDC9
disabled	фон #EBE5D9, текст #A5AAB3, курсор not-allowed
error (input)	рамка та підказка кольору #8B3A3A, ореол #F2DEDC

Окремо про focus. Його часто пропускають, а дарма: людина, яка працює з клавіатури, інакше просто не бачить, де вона зараз. Тому обведення в нас помітне, золотого кольору, і з невеликим відступом від самого елемента. Стани hover, active і focus додатково доступні як класи .is-hover, .is-active, .is-focus, тому в Storybook усі стани видно в одній таблиці без наведення миші.

Таблиця 8. Компоненти UI-kit у Storybook

Елемент	***Що показано в Storybook***	***Скріншот***
Кольори (tokens)	primary, secondary, accent, neutral, success, error, warning з hex-значеннями; перемикач теми	рис. 4
Типографіка	H1, H2, H3, body, small, caption, eyebrow	рис. 5
Button	primary, secondary, outline, ghost, danger; sm / md / lg; default, hover, active, focus, disabled	рис. 7
Input (у коді – Field)	default, hover, focus, error (з повідомленням), disabled; textarea для чату	рис. 8
Card	default, elevated, accent, interactive (hover), з блоком дій	рис. 9
TaskItem	звичайне, Top 3, виконане, перенесене; чекбокс, час, кнопка переносу	рис. 10
Navigation	меню шести розділів з активним, hover і вимкненим пунктами	рис. 11
ChatMessage	повідомлення користувача й асистента, індикатор «ШІ думає»	рис. 12
Badge, ProgressBar, Toast	нейтральний, gold, success, warning, error; прогрес зі станами; тост із дією Undo	рис. 13
BalanceWheel	радар-діаграма «Колесо балансу»: 8 сфер зі шкалою 1–9, на SVG з токенів	рис. 14

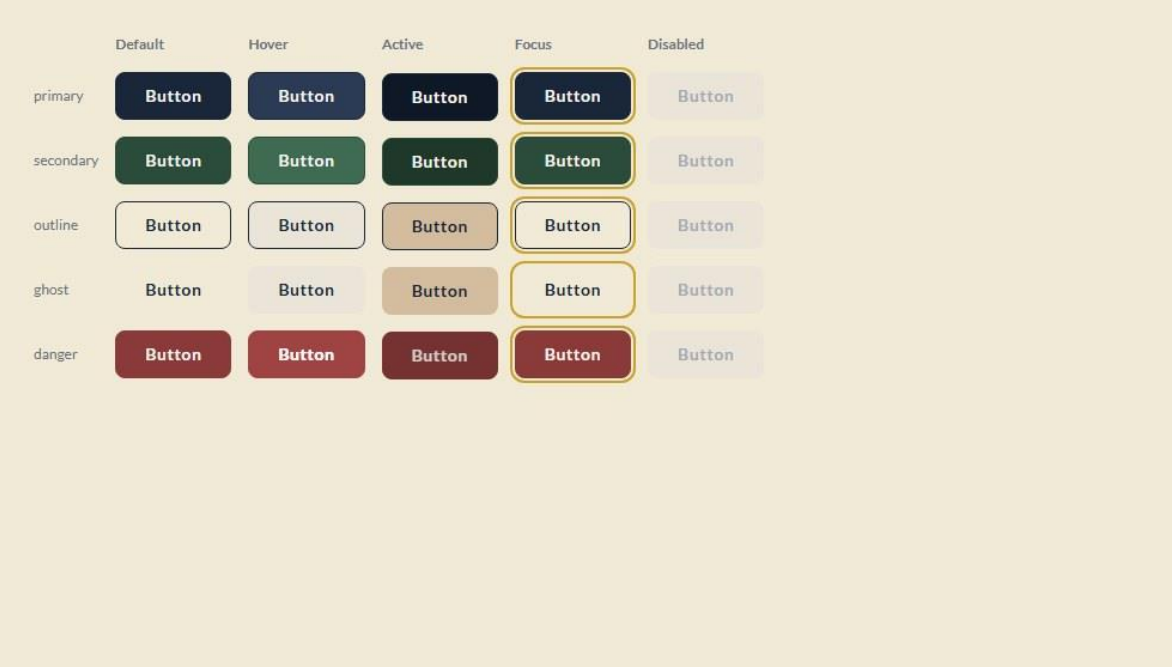


Рисунок 7. Button: п’ять варіантів у п’яти станах

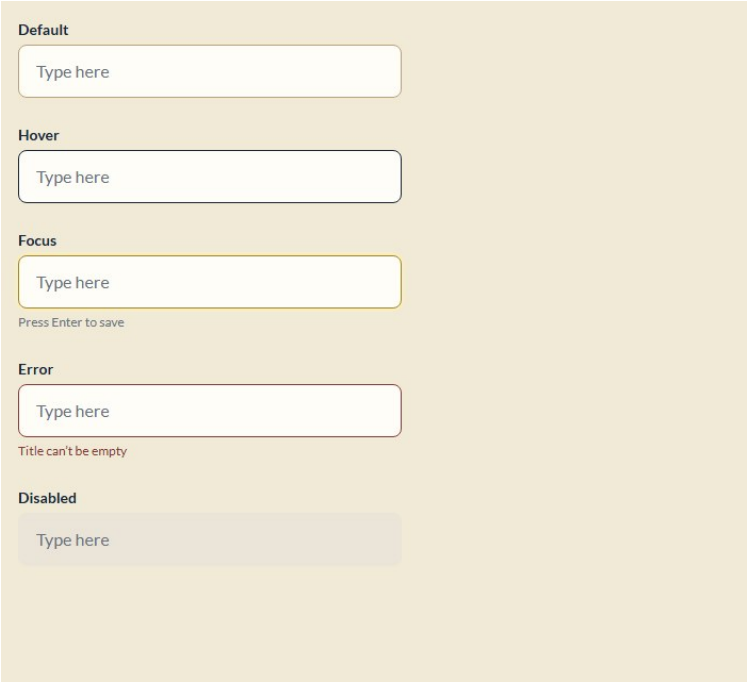


Рисунок 8. Input: default, hover, focus, error, disabled

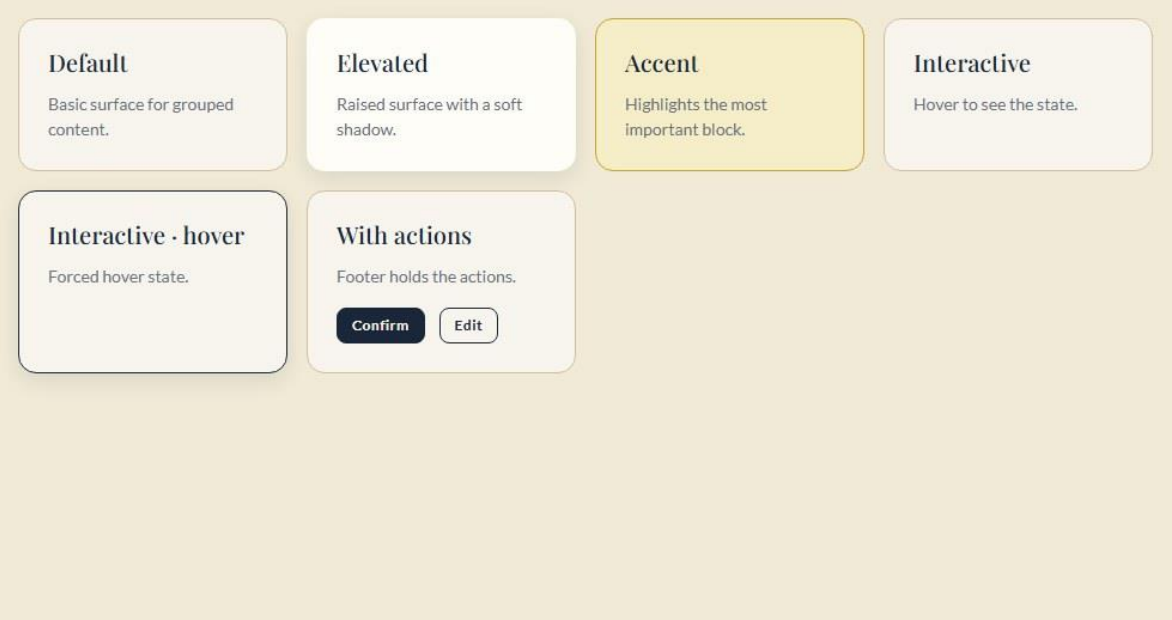


Рисунок 9. Card: варіанти та інтерактивний стан

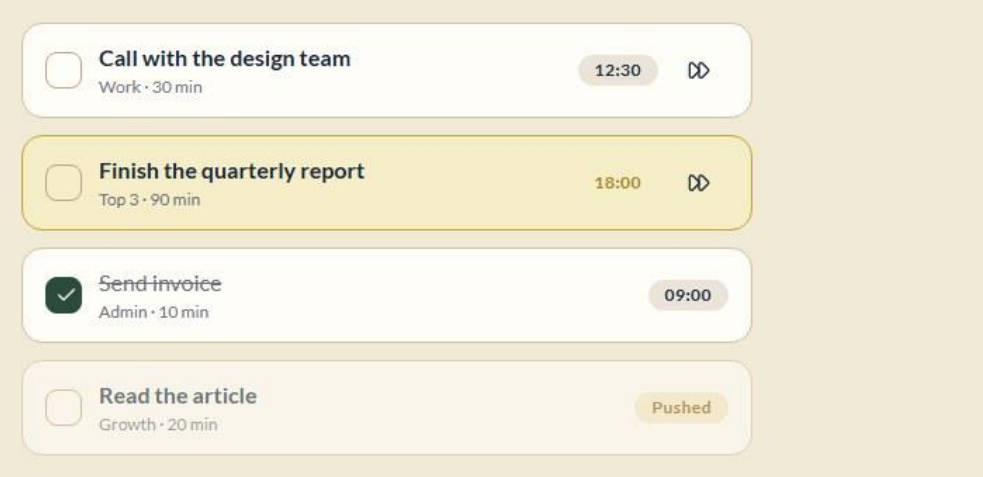


Рисунок 10. TaskItem: todo, Top 3, виконане, перенесене

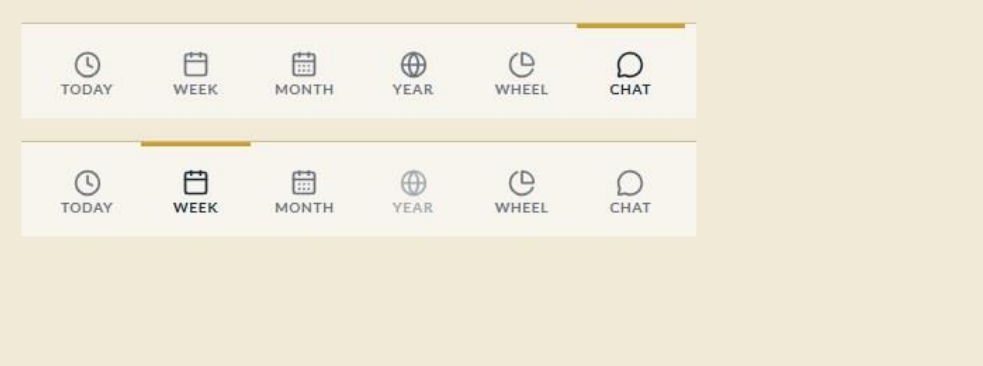


Рисунок 11. Navigation: активний і вимкнений пункти

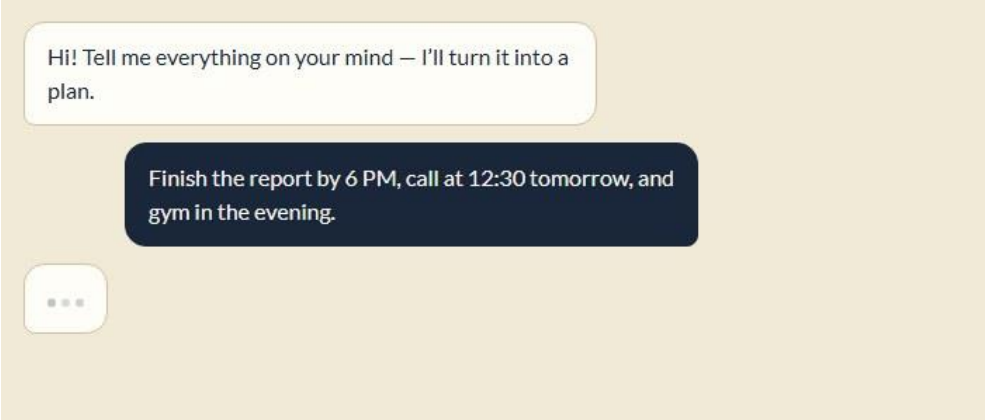


Рисунок 12. ChatMessage: користувач, асистент, індикатор набору



Badge

ProgressBar

Toast

Рисунок 13. Badge, ProgressBar та Toast



Рисунок 14. BalanceWheel: радар-діаграма на SVG

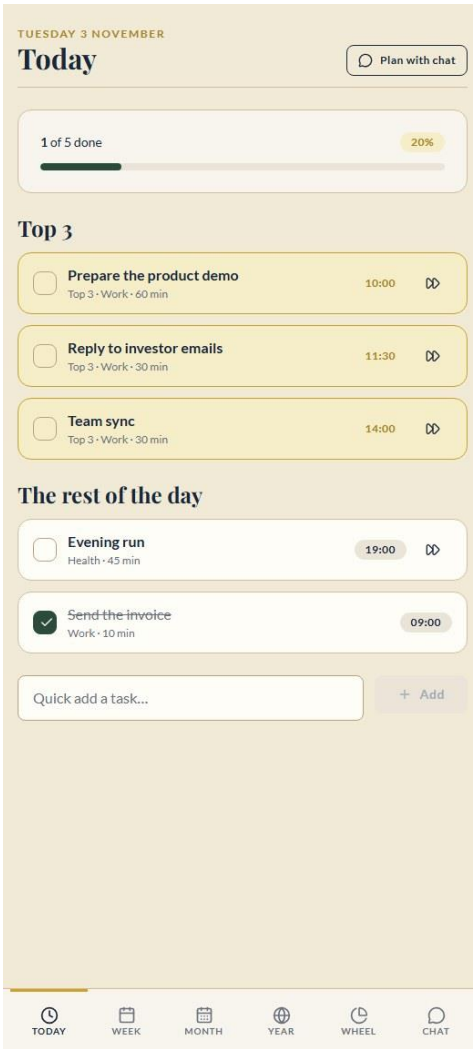
Реюзабельність: заміна кольору в одному файлі

Окремо виділимо головну вимогу до UI-kit – щоб змінити стиль, не доводилося переписувати екрани. Для цього в tokens.css є друга тема forest, яка перепризначає лише роль primary: темно-синій стає зеленим. Змінюються три рядки, а кнопки, активний пункт навігації, повідомлення

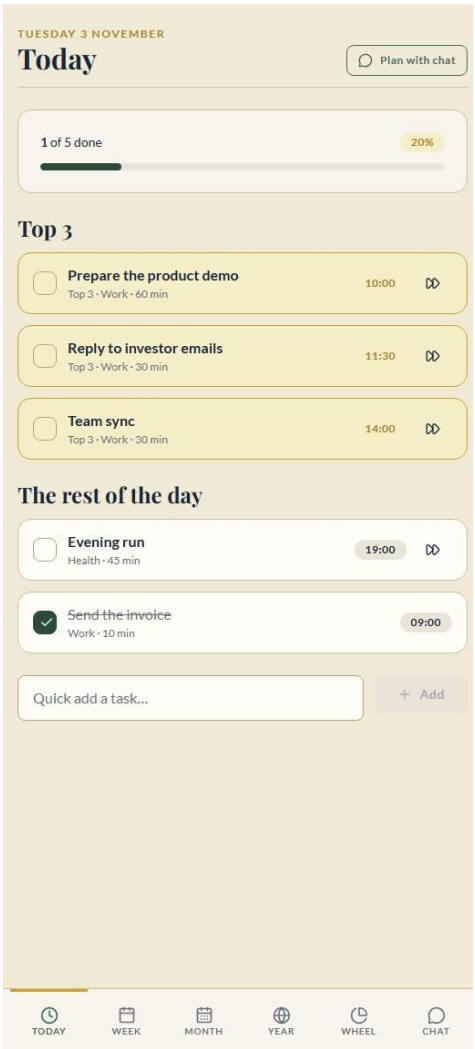
користувача в чаті й позначка сьогоднішнього дня в календарі перефарбовуються автоматично.

```
[data-theme='forest'] {
  --color-primary: var(--green-600);        /* було navy */
  --color-primary-hover: var(--green-800);  /* було navy-700 */
  --color-primary-active: var(--green-900); /* було navy-950 */
}
```

Тему можна перемикати прямо в тулбарі Storybook (Theme), тож заміну видно за секунди, а не за години.



Тема «Old Money» (primary – navy)



Тема «Forest» (primary – green)

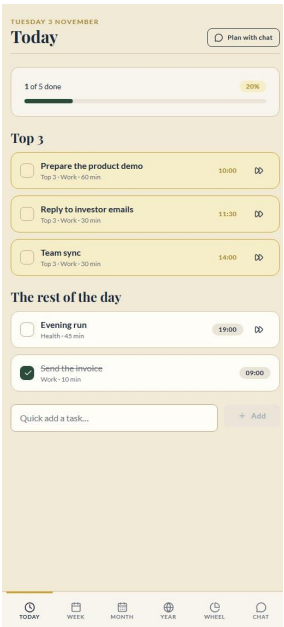
Рисунок 15. Той самий екран Today у двох темах без змін у компонентах

Екрани

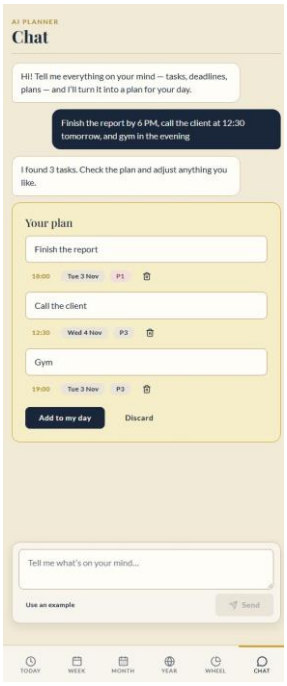
Екрани обрано за sitemap так, щоб разом вони покривали весь головний user flow. Важливий момент: кожен екран має складатися з компонентів UI-kit, а не стилізуватися окремо, – інакше сенс UI-kit просто губиться.

Таблиця 9. Ключові екрани

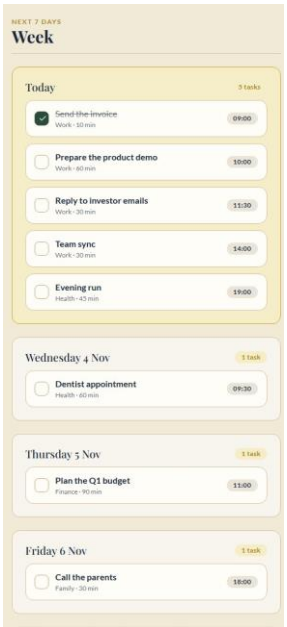
Екран (за sitemap)	***Компоненти UI-kit***	***Переходи***
Today	Card, ProgressBar, TaskItem, Input, Button, Badge, Toast, Navigation	→ Chat («Plan with chat»), перенесення завдання з Undo
Chat	ChatMessage, Card (картка плану), Input, Badge, Button	→ Today після підтвердження плану
Week	Card, TaskItem, Badge	Нижня навігація
Month (з Day)	Card, TaskItem	Вибір дня показує його завдання
Year	Card (interactive)	→ Month
Wheel of Balance	BalanceWheel, Card, Badge, Button	→ Vision & Mission, → Task Tracker, AI-інсайти
Vision & Mission	Input, Button, Toast	→ Wheel (Back)
Task Tracker	Card, ProgressBar, Badge	→ Wheel (Back)



Today



Chat (план від ІІІІ)



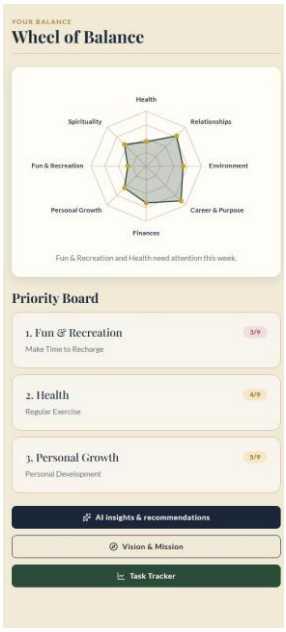
Week



Month



Year



Wheel of Balance



Vision & Mission



Task Tracker

Рисунок 16. Екрани прототипу (ширина телефона)

Проходження user flow у прототипі

Головний сценарій з Частини 2 проходиться в прототипі від початку до кінця. Усі переходи справжні, тобто це не клікабельні картинки, а працюючий код.

Таблиця 10. Сценарій «Чат → план дня» у прототипі

№	*Крок user flow*	*Що відбувається в прототипі*
1	Відкриття застосунку	Екран Today: прогрес дня, Top 3, решта справ. Якщо справ немає, показується порожній стан із кнопкою «Plan with chat»
2	Перехід у чат	Пункт Chat у навігації або кнопка «Plan with chat»
3	Ввід тексту	Поле повідомлення; кнопка «Use an example» підставляє приклад із PRD
4	ШІ аналізує	Індикатор «ШІ думає» близько секунди
5	Завдання розпізнано?	Ні – відповідь «What exactly needs to be done?» і повтор вводу. Так – картка плану
6	План влаштовує?	Назви редагуються просто в картці, зайве видаляється; порожня назва дає помилку поля й блокує підтвердження
7	Підтвердження	«Add to my day» → перехід на Today, повідомлення «3 tasks added to your day»

№	*Крок user flow*	*Що відбувається в прототипі*
8	Кінцеві стани	Виконано (чекбокс, закреслення, оновлений прогрес) або перенесено на наступний слот із повідомленням і дією Undo

Один важливий нюанс. Розбір повідомлення ШІ в прототипі імітується простими правилами (файл src/data/parse.js): текст ділиться за комами та словами «and» і «then», з нього витягуються час, слово «tomorrow» і ключові слова для категорії. Справжню модель буде підключено в ПЗ № 3 через наявний ШІ-агент застосунку, а інтерфейс і всі переходи при цьому не зміняться. Стан «помилка мережі» з user flow покрито компонентом Toast у варіанті error, але в прототипі він не викликається, бо мережі тут немає.

Межі прототипу та відхилення від PRD

Щоб не було розбіжностей зі специфікацією з ПЗ № 1, прямо скажемо, чого в прототипі ще немає. Не реалізовано: нагадування, перепланування через чат («перенеси тренування на завтра»; у прототипі є лише перенесення кнопкою на наступний слот з Undo), справжній ШІ, уточнювальне запитання при неоднозначній даті, реєстрація й онбординг. Усе це залежить від підключення моделі, тому це ПЗ № 3.

Інтерфейс прототипу англомовний, як і наявний застосунок: назви розділів і сфер Колеса балансу в ньому англійські. З цієї ж причини імітація розбору в parse.js розуміє англійські слова («and», «then», «tomorrow»), а українські приклади з PRD вона не розбере. Українську мову інтерфейсу й розбору заплановано разом зі справжнім ШІ.

І навпаки, дещо виходить за межі MVP з PRD: екрани Week, Month, Year і Task Tracker та швидке додавання завдання (Quick Add) у PRD віднесено до наступних версій, але вони вже були у вашому застосунку, тому ми показали їх у прототипі, щоб sitemap відповідав реальній навігації.

Бонус: мобільна адаптація

Інтерфейс побудовано mobile-first: контент має максимальну ширину 720 px і центрується, нижня навігація фіксована, календар і сітки перебудовуються без горизонтального скролу. Усі екрани на рисунку 16 показано саме на ширині телефона, а на широкому екрані той самий код виглядає так:

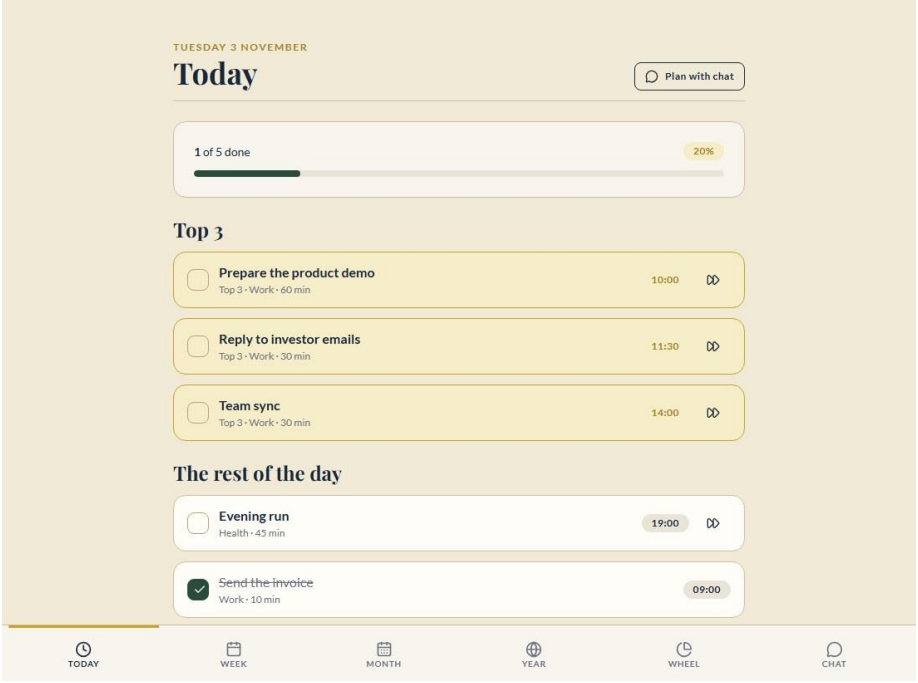


Рисунок 17. Екран Today на широкому екрані (той самий код, без окремої версії)

Висновок

Друге практичне завдання стало для нас моментом, коли Cassiopeia перестала бути лише текстом. Ми порівняли інструменти – від класичної Figma до AI-інструментів, що працюють з кодом, – і свідомо обрали шлях через код, бо так робота над прототипом не пропаде в ПЗ № 3. Побудували трирівневий sitemap та два user flow, де кожне розгалуження спирається на UX-принципи з PRD. Зібрали мудборд і вже з нього вивели палітру, шрифт і форму елементів. У Частині 4 ці рішення перетворилися на UI-kit у коді: design tokens в одному файлі, десять компонентів зі станами в Storybook і вісім екранів із робочою навігацією, зібраних лише з компонентів. Вимогу

реюзабельності підтверджує друга тема: щоб змінити головний колір, достатньо трьох рядків у tokens.css.